MediSpark — Project Report

Amrita Vishwa Vidyapeetham, Coimbatore

Course: 23AID302 — Big Data Analytics | **Faculty:** Sreeja B P

Team Members:

Name	Register No.
Suman Maitreya M	CB.AI.U4AID24156
Rajkumar R	CB.AI.U4AID24143
Harish K	CB.AI.U4AID24118
GRS Kartheek	CB.AI.U4AID24116

A distributed multi-disease clinical risk prediction engine built on Apache Spark and Hadoop HDFS, simultaneously predicting patient risk for four diseases and scoring new patients in real time.

Abstract

MediSpark is a distributed clinical risk prediction system built on Apache Spark 3.5.1 and Hadoop HDFS 3.2.1, implemented in Scala 2.12.18. The system simultaneously predicts patient risk for four diseases — heart disease, diabetes, stroke, and lung cancer — using Random Forest classifiers trained on SMOTE-augmented datasets of 500,000 records each, and classifies chest X-ray images as normal or abnormal using a Gradient-Boosted Tree (GBT) classifier with 16 handcrafted pixel features.

The entire infrastructure runs on a Dockerized cluster comprising a Spark master, two worker nodes, one HDFS NameNode, and three DataNodes. A seven-stage pipeline handles end-to-end processing: data ingestion, automated cleaning, feature engineering (StringIndexer, VectorAssembler, StandardScaler), model training, evaluation, X-ray classification, and real-time streaming via Spark Structured Streaming with micro-batch scoring every 10 seconds.

Evaluation on held-out test sets (95,670–100,000 samples per disease) yielded 99.60% accuracy for heart disease, 97.45% for diabetes, 92.13% for stroke, and 93.57% for lung cancer. A leakage-safe train/test split strategy — tagging samples during SMOTE

augmentation rather than randomly re-splitting — ensured honest evaluation. The streaming module loads all four trained models and scores incoming patient records across all diseases simultaneously, joining predictions by patient ID and writing results as Parquet files.

Introduction

Problem Statement

Clinical decision-making relies on timely risk assessment across multiple conditions, yet most existing systems evaluate diseases in isolation. A patient presenting with elevated glucose, high blood pressure, and a history of smoking faces simultaneous risk for heart disease, diabetes, stroke, and lung cancer — but conventional tools require separate models, separate data pipelines, and separate clinical workflows for each. This fragmentation delays diagnosis and increases the cognitive burden on clinicians.

Motivation

Apache Spark's distributed computing framework offers the throughput and fault tolerance needed to process large-scale medical datasets. Combined with Hadoop HDFS for distributed storage and Spark MLlib for scalable machine learning, it becomes possible to build a single unified pipeline that ingests, cleans, engineers features for, trains, evaluates, and deploys models for multiple diseases — all within one coordinated system.

Objectives

1. Build a distributed data pipeline on Spark and HDFS that processes four tabular disease datasets and one medical imaging dataset end to end
2. Train Random Forest classifiers for heart disease, diabetes, stroke, and lung cancer on SMOTE-augmented data with leakage-safe train/test splits
3. Classify chest X-ray images using a GBT model with handcrafted pixel features, without requiring deep learning frameworks
4. Deploy real-time scoring via Spark Structured Streaming, predicting all four diseases simultaneously for each incoming patient record
5. Containerize the entire cluster using Docker Compose for reproducible deployment

Course Context

This project was developed for **23AID302 — Big Data Analytics** by Suman Maitreya M, Rajkumar R, and Harish.

Literature Survey

Base Paper

AlMohimeed A., "Cloud-based real-time enhancement for disease prediction using Confluent Cloud, Apache Kafka, feature optimization, and explainable artificial intelligence," *PeerJ Computer Science*, vol. 11, e2899, 2025. DOI: 10.7717/peerj-cs.2899

The base paper by AlMohimeed (2025) proposes a two-phase cloud-based system for chronic kidney disease (CKD) prediction. The offline phase employs a stacking ensemble — combining Logistic Regression, Decision Tree, Random Forest, SVM, and Naive Bayes as base learners with a meta-learner — alongside Genetic Algorithm (GA) and Particle Swarm Optimization (PSO) for feature selection on the UCI CKD dataset (400 samples, 24 features). SHAP (SHapley Additive exPlanations) provides global and local explainability, identifying Hemoglobin as the most influential predictor. The online phase uses Confluent Cloud and Apache Kafka (Producer API → topic → Consumer API) for real-time streaming predictions. The stacking model with GA-selected features achieved 100% accuracy, precision, recall, and F1-score on a 20% test split, and 98.75% on a 40% split.

MediSpark extends this work in several directions: (1) simultaneous multi-disease prediction across four tabular diseases plus chest X-ray binary classification, versus the base paper's single-disease CKD focus; (2) processing over 2 million records on a fully local Docker-based Spark 3.5.1 / HDFS 3.2.1 cluster, eliminating dependency on commercial cloud services like Confluent Cloud; (3) Spark Structured Streaming with 5-second micro-batches for real-time scoring, replacing the Kafka-based pipeline; and (4) integration of medical imaging through handcrafted radiological features and Gradient-Boosted Trees — a direction the base paper itself identifies as future work.

Related Work

Disease prediction with machine learning. Numerous studies have applied Random Forest, SVM, and logistic regression to individual disease datasets. The UCI Heart Disease dataset and the Pima Indians Diabetes dataset are among the most studied benchmarks

in clinical ML. Most prior work trains a single model per disease, evaluates on a random split of the original (often small) dataset, and does not address deployment or real-time scoring.

Big Data frameworks in healthcare. Apache Spark has been applied to genomics, electronic health record (EHR) analysis, and medical image processing. Spark MLlib provides scalable implementations of Random Forest, GBT, and pipeline utilities (StringIndexer, VectorAssembler, StandardScaler) that eliminate the need for manual feature-vector construction.

Medical image classification without deep learning. While CNNs dominate chest X-ray classification, Spark MLlib does not natively support convolutional layers. Prior work has explored handcrafted features (texture descriptors, histogram statistics, edge features) combined with traditional classifiers as a practical alternative when deep learning infrastructure is unavailable.

Gaps addressed by MediSpark. MediSpark differs from prior work in three ways: (1) it predicts four diseases simultaneously through a single unified pipeline rather than isolated models, (2) it uses a leakage-safe SMOTE augmentation strategy that tags train/test membership before augmentation to prevent synthetic-neighbor leakage, and (3) it deploys real-time multi-disease scoring via Structured Streaming.

System Architecture

Technology Stack

Component	Version / Detail
Language	Scala 2.12.18
Compute engine	Apache Spark 3.5.1 (Spark MLlib)
Storage	Hadoop HDFS 3.2.1
Containerization	Docker Compose
Serving (X-ray)	Flask + ONNX Runtime
Build tool	sbt

Docker Cluster Topology

The cluster is deployed via Docker Compose with the following containers:

- **spark-master** — orchestrates job scheduling and resource allocation
- **spark-worker-1** and **spark-worker-2** — each configured with 2 CPU cores and 2 GB RAM
- **namenode** — HDFS NameNode managing the filesystem namespace at `hdfs://namenode:9000`
- **datanode-1**, **datanode-2**, **datanode-3** — three HDFS DataNodes providing distributed block storage

All Spark jobs run with `--driver-memory 1g --executor-memory 1g` to respect the 2 GB per-worker constraint.

Seven-Stage Pipeline

The system executes a sequential seven-stage pipeline, each stage reading from and writing to HDFS:

1. **Ingestion** — loads raw CSV datasets into HDFS at `/medispark/ingested/`
2. **Cleaning** — handles nulls, drops unnecessary columns, standardizes column names, casts types, removes outliers; writes to `/medispark/cleaned/`
3. **Feature Engineering** — applies StringIndexer for categorical columns, VectorAssembler to combine features, and StandardScaler for normalization; writes to `/medispark/features/`
4. **Model Training** — trains Random Forest classifiers (one per disease) on the training split; saves models to `/medispark/models/`
5. **Streaming Simulation** — Structured Streaming with file source, scoring micro-batch CSVs against all four trained models simultaneously
6. **Model Evaluation** — computes accuracy, F1, precision, recall, AUC-ROC, confusion matrices, feature importances on held-out test sets
7. **Chest X-ray Classification** — trains a GBT classifier on 16 handcrafted pixel features extracted from chest X-ray images

Spark Execution Model

Spark's Catalyst optimizer compiles DataFrame operations into an optimized DAG of RDD transformations. During the actual training run, the Spark UI recorded 84 completed jobs, 161 completed stages, and 25 skipped stages. MLlib's Random Forest internally caches bagged tree points as an RDD (303.6 MiB, 2 partitions, Memory Deserialized 1x Replicated). Shuffle write averaged 259.1 KiB per stage.

Datasets

Five datasets power the system — four tabular clinical datasets and one chest X-ray image dataset.

Tabular Datasets

Dataset	Source	Original size	After SMOTE	Features	Target
Heart Disease	UCI / Kaggle	~920 rows	500,000	13 (age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal)	Binary (0/1)
Diabetes	Pima Indians / Kaggle	768 rows	500,000	8 (Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age)	Binary (0/1)
Stroke	Kaggle	~5,110 rows	500,000	10 (5 numeric + 5 categorical: gender, ever_married, work_type, Residence_type, smoking_status)	Binary (0/1)
Lung Cancer	Kaggle	~1,000 rows	500,000	23 (Age, Gender, Air_Pollution, Alcohol_use, and 19 symptom/risk factors)	Multi-class (Low / Medium / High)

All four tabular datasets were augmented to 500,000 rows using SMOTE (Synthetic Minority Over-sampling Technique). Critically, the train/test split tag (`_split`) was assigned during the SMOTE augmentation process itself — synthetic neighbors were generated separately for each split — ensuring that no synthetic sibling of a training sample appears in the test set. This leakage-safe strategy prevents the inflated accuracy that would result from a random re-split of already-augmented data.

Chest X-ray Dataset

The chest X-ray dataset consists of 5,606 images sourced from Kaggle (Chest X-ray Pneumonia dataset), labeled as Normal or Abnormal. Since Spark MLlib does not support convolutional neural networks, 14 handcrafted pixel-level features were extracted from each 128x128 grayscale image:

- **Pixel statistics:** pixel_mean, pixel_std, pixel_skew, pixel_kurtosis, pixel_iqr, pixel_cv
- **Spatial features:** center_mean, periphery_ratio
- **Edge and texture:** edge_density, gradient_mag_mean, entropy
- **Symmetry:** symmetry_lr_mean
- **Profile features:** row_profile_std, col_profile_std

Two demographic features (patient_age, patient_gender) were appended, bringing the total to 16 features per image.

Implementation

The project source lives under `medispark/src/main/scala/medispark/` with one package per pipeline stage. Key implementation details are presented below with excerpts from the actual source code.

Stage 2: Data Cleaning (`cleaning/DataCleaning.scala`)

Each disease dataset receives tailored cleaning. For stroke data, null BMI values are filled with the column median computed via `approxQuantile` , and missing smoking status is filled with "Unknown":

```
val bmiMedian = strokeDf
  .stat.approxQuantile("bmi", Array(0.5), 0.01)(0)
strokeDf = strokeDf.na.fill(bmiMedian, Seq("bmi"))
strokeDf = strokeDf.na.fill("Unknown", Seq("smoking_status"))
```

Heart disease cleaning filters outliers on cholesterol and resting blood pressure. Diabetes cleaning removes physiologically impossible zero values for Glucose, BloodPressure, and BMI. Lung cancer cleaning drops index and Patient_Id columns and renames the `Level` column to `label_str` for downstream indexing. All datasets standardize the target column to `label` .

Stage 3: Feature Engineering (`features/FeatureEngineering.scala`)

Categorical columns in the stroke dataset are encoded using `StringIndexer` with `handleInvalid("keep")` to gracefully handle unseen categories at inference time:

```
val catCols = Array("gender", "ever_married", "work_type",
                    "Residence_type", "smoking_status")
val indexedCols = catCols.map(_ + "_idx")

val indexers = catCols.zip(indexedCols).map { case (in, out) =>
  new StringIndexer()
    .setInputCol(in).setOutputCol(out)
    .setHandleInvalid("keep")
}
```

A reusable `assembleAndScale` helper casts all feature columns to `DoubleType`, assembles them into a vector with `VectorAssembler(handleInvalid = "skip")`, and normalizes with `StandardScaler(withStd = true, withMean = true)`. The output retains only `label`, `features`, and the `_split` tag for leakage-safe partitioning.

For lung cancer (a three-class problem), the string target `label_str` (Low / Medium / High) is converted to a numeric label via an additional `StringIndexer`.

Stage 4: Model Training (`training/ModelTraining.scala`)

Random Forest classifiers are configured per disease:

```
val numTrees = disease match {
  case "lung_cancer" => 150
  case _             => 100
}

val rf = new RandomForestClassifier()
  .setLabelCol("label")
  .setFeaturesCol("features")
  .setNumTrees(numTrees)
  .setMaxDepth(10)
  .setMinInstancesPerNode(5)
  .setSeed(42L)
  .setFeatureSubsetStrategy("sqrt")
```

The training loop iterates over `Seq("heart", "diabetes", "stroke", "lung_cancer")`, loading each disease's feature-engineered Parquet from HDFS. The leakage-safe split filters on the `_split` column — `data.filter(data("_split") === "train")` and `data.filter(data("_split") === "test")` — rather than calling `randomSplit`, which would scatter near-duplicate synthetic siblings across train and test. Trained models are saved to HDFS with `model.write.overwrite().save()`.

ML Pipeline and Results

Pipeline Stages

Every disease model follows the same four-stage ML pipeline within Spark MLlib:

1. **StringIndexer** — encodes categorical string columns as numeric indices (used for stroke's 5 categorical features and lung cancer's multi-class target)
2. **VectorAssembler** — combines all feature columns into a single `features` vector, with `handleInvalid = "skip"` to drop rows with null or invalid values
3. **StandardScaler** — normalizes the feature vector to zero mean and unit standard deviation (`withStd = true, withMean = true`)
4. **RandomForestClassifier / GBTClassifier** — the final estimator, configured per disease

Algorithm Configuration

Parameter	Tabular (Heart, Diabetes, Stroke)	Tabular (Lung Cancer)	X-ray (GBT)
Algorithm	Random Forest	Random Forest	GBT Classifier
Trees / Iterations	100	150	200
maxDepth	10	10	6
featureSubsetStrategy	sqrt	sqrt	—
stepSize	—	—	0.1
subsamplingRate	—	—	0.8
minInstancesPerNode	5	5	5

Parameter	Tabular (Heart, Diabetes, Stroke)	Tabular (Lung Cancer)	X-ray (GBT)
seed	42	42	42

Evaluation Results (Stage 6)

All metrics are computed on held-out test sets using the leakage-safe `_split` tag. No test sample or its synthetic neighbor appears in training.

Disease	Test samples	Accuracy	F1 Score	Precision	Recall	AUC-ROC
Heart Disease	100,000	99.60%	99.60%	99.60%	99.60%	99.99%
Diabetes	95,670	97.45%	97.44%	97.47%	97.45%	99.70%
Stroke	99,998	92.13%	89.55%	92.75%	92.13%	94.57%
Lung Cancer	100,000	93.57%	93.57%	93.58%	93.57%	N/A (3-class)

Confusion Matrices

Heart Disease (TP = 48,271 | TN = 51,328 | FP = 349 | FN = 52) — near-perfect separation with only 401 misclassifications out of 100,000.

Diabetes (TP = 37,007 | TN = 56,223 | FP = 616 | FN = 1,824) — strong performance; most errors are false negatives (missed diabetic patients).

Stroke (TP = 1,714 | TN = 90,413 | FP = 2 | FN = 7,869) — the model achieves 99.88% precision on the positive class but only 17.89% recall, reflecting extreme class imbalance even after SMOTE. The model is conservative: it rarely flags a healthy patient as at-risk, but misses most true stroke cases.

Lung Cancer (3-class confusion matrix):

	Pred Low	Pred Medium	Pred High
Actual Low	38,446	1,270	932
Actual Medium	956	32,616	934
Actual High	923	1,411	22,512

Per-class F1 scores: Low = 94.96%, Medium = 93.45%, High = 91.47%.

Real-Time Streaming

Architecture

Stage 5 implements a real-time scoring simulation using Spark Structured Streaming with a file source. Incoming micro-batch CSV files (representing new patient records arriving at a hospital) are picked up from `hdfs://namenode:9000/medispark/stream_input/`, scored against all four trained models, and written as Parquet to `hdfs://namenode:9000/medispark/stream_output/`.

Stream Schema

Each incoming patient record contains 10 common clinical fields that are mapped to each disease model's specific feature vector:

Field	Type	Description
patient_id	String	Unique identifier for join
timestamp	String	Arrival time
age	Double	Patient age
gender	Double	Gender (encoded)
bmi	Double	Body mass index
blood_pressure	Double	Resting blood pressure
cholesterol	Double	Serum cholesterol
glucose	Double	Blood glucose level
smoking	Double	Smoking status
exercise	Double	Exercise indicator
hypertension	Double	Hypertension flag
heart_disease_history	Double	Prior heart disease

Per-Disease Feature Mapping

The streaming module maps the 10 generic fields to each disease's specific feature schema via `withColumn` transformations:

- **Heart (13 features):** age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal — with derived fields like `fbs = when(glucose > 120, 1.0).otherwise(0.0)` and `thalach = 150.0 - (age * 0.5)`
- **Diabetes (8 features):** Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI_d, DiabetesPedigreeFunction, Age_d
- **Stroke (10 features):** 5 numeric (age, hypertension, heart_disease, avg_glucose_level, bmi) + 5 indexed (gender_idx, ever_married_idx, work_type_idx, Residence_type_idx, smoking_status_idx)
- **Lung Cancer (23 features):** Age_l, Gender_l, and 21 symptom/risk factor columns

Scoring and Join

Each micro-batch is scored by all four models independently, then the predictions are joined by `patient_id`:

```
val scored = heartPreds
  .join(diabetesPreds, "patient_id")
  .join(strokePreds, "patient_id")
  .join(lungPreds, "patient_id")
scored.write.mode("append")
  .parquet(s"$streamOut/batch_$batchId")
```

The stream triggers every 10 seconds (`ProcessingTime("10 seconds")`) and runs for 2 minutes (`awaitTermination(120000)`). The output for each patient is a single row containing the original demographics plus four disease risk predictions with probability vectors.

Chest X-ray Classification

Approach

Stage 7 classifies chest X-ray images as Normal or Abnormal using a Gradient-Boosted Tree (GBT) classifier trained on handcrafted pixel features. This approach was chosen because Spark MLlib does not natively support convolutional neural networks, and the memory-constrained Docker cluster (1 GB executor memory) cannot accommodate deep learning workloads.

Feature Extraction

14 handcrafted features were extracted from each 128x128 grayscale X-ray image and stored as Parquet in HDFS. These features capture pixel-level statistics, spatial patterns, texture, and symmetry:

Category	Features
Pixel statistics	pixel_mean, pixel_std, pixel_skew, pixel_kurtosis, pixel_iqr, pixel_cv
Spatial	center_mean, periphery_ratio
Edge / texture	edge_density, gradient_mag_mean, entropy
Symmetry	symmetry_lr_mean
Profile	row_profile_std, col_profile_std
Demographics	patient_age, patient_gender

GBT Pipeline

The ML pipeline consists of VectorAssembler, StandardScaler, and GBTClassifier:

```
val gbt = new GBTClassifier()
  .setLabelCol("label")
  .setFeaturesCol("features")
  .setMaxIter(200)
  .setMaxDepth(6)
  .setStepSize(0.1)
  .setSubsamplingRate(0.8)
  .setMinInstancesPerNode(5)
  .setSeed(42)
```

The 5,606 images are split 80/20. The training set is balanced by oversampling the minority class (Abnormal) with replacement to match the majority class count.

Results

The model achieves strong binary classification performance on the test set. Accuracy, F1, precision, recall, and AUC-ROC metrics are computed using Spark's `MulticlassClassificationEvaluator` and `BinaryClassificationEvaluator`. The

model and evaluation metrics (as JSON) are saved to HDFS at `/medispark/models/xray_model` and `/medispark/evaluation/xray` respectively.

Challenges, Limitations, and Future Work

Challenges Faced

Memory constraints. Each Spark worker has only 2 GB RAM, and jobs must run with `--driver-memory 1g --executor-memory 1g`. This required careful caching strategies and limited the complexity of models that could be trained. Random Forest's internal caching of bagged tree points (303.6 MiB) consumed a significant fraction of available memory.

No CNN support in Spark MLlib. Chest X-ray classification typically relies on deep convolutional neural networks (ResNet, DenseNet). Since Spark MLlib offers no CNN implementation, 16 handcrafted pixel features were engineered as a practical alternative — covering pixel statistics, spatial patterns, edge density, entropy, and symmetry.

Stroke class imbalance. Despite SMOTE augmentation to 500,000 rows, the stroke model's positive-class recall remained at only 17.89%. The model achieves 99.88% precision on stroke predictions (nearly zero false positives) but misses the majority of true stroke cases, indicating that SMOTE alone is insufficient for this level of imbalance.

Limitations

SMOTE-augmented accuracy. The high accuracy figures (99.60% for heart disease) may partially reflect patterns in synthetic data rather than genuine clinical signal. Validation on real-world clinical datasets of comparable size would be needed to confirm generalization.

Single-node Docker cluster. Although the system uses distributed primitives (HDFS replication, Spark workers), all containers run on a single host machine. This demonstrates the architecture but does not test true network-distributed fault tolerance or data locality.

Streaming defaults. The streaming module maps 10 generic fields to each disease's feature schema using hardcoded defaults (e.g., `lit(0.0)` for chest pain type, `lit(150.0) - (age * 0.5)` for max heart rate). In a production deployment, these would need to be replaced with actual clinical measurements.

Future Work

- **Cloud deployment** on AWS EMR or Azure HDInsight for true multi-node scalability and fault tolerance
- **Deep learning for X-ray classification** via ONNX export and TorchServe integration, enabling CNN-based models (ResNet-50, DenseNet-121) to be served alongside Spark
- **Apache Kafka integration** to replace the file-based streaming source with a real message queue for hospital EHR systems
- **Model explainability** using SHAP (SHapley Additive exPlanations) or LIME to provide clinicians with interpretable risk factors per prediction
- **Threshold tuning for stroke** — adjusting the classification threshold to improve positive-class recall at the expense of some precision, guided by clinical cost-sensitivity analysis

Conclusion

MediSpark demonstrates that a single distributed system on Apache Spark and Hadoop HDFS can ingest, clean, engineer features for, train, evaluate, and deploy models for multiple diseases within one coordinated pipeline. The system achieves 99.60% accuracy on heart disease, 97.45% on diabetes, 92.13% on stroke, and 93.57% on lung cancer using Random Forest classifiers, while a GBT model classifies chest X-ray images using 16 handcrafted features without any deep learning dependency.

The leakage-safe train/test split strategy — tagging splits during SMOTE augmentation rather than after — ensures honest evaluation and is a methodological contribution applicable to any SMOTE-based pipeline. The Structured Streaming module proves that all four models can score incoming patients simultaneously in real time, joining predictions by patient ID within 10-second micro-batches.

The primary limitation is the stroke model's low positive-class recall (17.89%), which highlights that SMOTE augmentation alone cannot fully resolve extreme class imbalance. Future work should explore cost-sensitive learning, threshold tuning, and real-world clinical validation. Deploying on a true multi-node cloud cluster (AWS EMR or Azure HDInsight) and integrating Apache Kafka for live hospital data streams would bring MediSpark closer to production readiness.

References

Base Paper

AlMohimeed A., "Cloud-based real-time enhancement for disease prediction using Confluent Cloud, Apache Kafka, feature optimization, and explainable artificial intelligence," *PeerJ Computer Science*, vol. 11, e2899, 2025. DOI: 10.7717/peerj-cs.2899

Datasets

1. UCI Heart Disease — Kaggle (link)
2. Pima Indians Diabetes — Kaggle (link)
3. Stroke Prediction — Kaggle (link)
4. Lung Cancer Prediction — Kaggle (link)
5. Chest X-ray (Normal/Abnormal) — Kaggle (link)

Technologies

- Apache Spark 3.5.1 — spark.apache.org
- Hadoop HDFS 3.2.1 — hadoop.apache.org
- Scala 2.12.18 — scala-lang.org
- Spark MLlib — spark.apache.org/mllib
- Docker Compose — docs.docker.com/compose

Team: Suman Maitreya M , Rajkumar R , Harish K , GRS Kartheek

Course: 23AID302 — Big Data Analytics